

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	Course Code: CMPE-112L
Assignment Type: Lab Report	Dated: July 1, 2026
Semester: 1st Semester	Session: Spring 2026
Lab/Project/Assignment #: 1	CLOs to be covered: CLO1
Lab Title: Input/Output, Data Types, Operators, Conditional Statements (If/Else)	Teacher Name: Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems.					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
-------	-------	-------	--------

Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 1

Lab Goals / Objectives:

- Understand the basic structure of a Python program
- Learn to use the print() function for output with formatting options
- Understand Python data types: int, float, bool, string
- Learn to use the input() function for user input
- Apply sep and end parameters in print() for custom formatting
- Use type() function to identify variable data types

Equipment Required:

- Computer System with Python 3.x installed
- Text Editor or IDE (PyCharm, VS Code, or IDLE)
- Operating System: Windows / Linux / macOS

Theory:

1. The print() Function

The `print()` function in Python is used to output data to the standard output device (screen). It is one of the most frequently used built-in functions in Python. The basic syntax is:

```
print(value1, value2, ..., sep=' ', end='\n')
```

The `print()` function converts the arguments to strings, writes them to the console, and adds a newline at the end by default. It can take multiple arguments separated by commas.

2. The sep Parameter

The `sep` parameter in `print()` specifies how to separate multiple values. By default, it is a space character ' '. You can customize it to any string:

```
print('UET', 'Lahore', sep='-') # Output: UET-Lahore
```

3. The end Parameter

The `end` parameter specifies what to print at the end of the output. By default, it is a newline character '\n'. Changing it allows continuing output on the same line:

```
print('Hello', end=' ')\nprint('World') # Output: Hello World
```

4. Escape Sequences

Escape sequences are special characters that begin with a backslash (\) and have special meaning in strings:

- `\n` - Newline: Moves the cursor to the next line
- `\t` - Tab: Inserts a horizontal tab
- `\\` - Backslash: Prints a literal backslash
- `\'` - Single quote: Prints a single quote

- \" - Double quote: Prints a double quote

5. The input() Function

The input() function allows user input from the keyboard. It always returns a string value, which must be converted to the desired data type using type casting functions:

```
name = input('Enter your name: ')
age = int(input('Enter your age: '))
```

6. Python Data Types

Python has several built-in data types:

- int (Integer): Whole numbers without decimal points. Example: x = 10
- float (Floating-point): Numbers with decimal points. Example: y = 3.14
- bool (Boolean): Logical values True or False. Example: z = True
- str (String): Sequence of characters enclosed in quotes. Example: name = 'UET'

7. Type Casting

Type casting is the process of converting one data type to another:

- int() - Converts a value to an integer
- float() - Converts a value to a floating-point number
- str() - Converts a value to a string
- bool() - Converts a value to a boolean

8. The type() Function

The type() function returns the data type of a variable or value:

```
x = 10
print(type(x)) # Output: <class 'int'>
```

9. String Formatting with f-strings

f-strings (formatted string literals) provide a concise way to embed expressions inside string literals using curly braces:

```
name = 'Hashir'
print(f'Name: {name}')
```

Lab Work:

Task 1: Print Your Information

Write a Python program that prints your personal information including name, department, and university using the print() function.

Code Summary:

This task demonstrates the basic usage of the print() function to display multiple lines of information. Each print() statement outputs a line of text containing personal details.

Key Code Lines:

```
# Task 1: Print Your Information
print("Name: Hashir Zahid")
print("Department: Computer Engineering")
print("University: UET Lahore")
```

Output:



```
Run Task1_Print_Information x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week 1\Task1_Print_Information.py"
Name: Hashir Zahid
Department: Computer Engineering
University: UET Lahore
Process finished with exit code 0
```

Task 2: Using sep and end Parameters

Write a program that demonstrates the use of sep and end parameters in the print() function to customize output formatting.

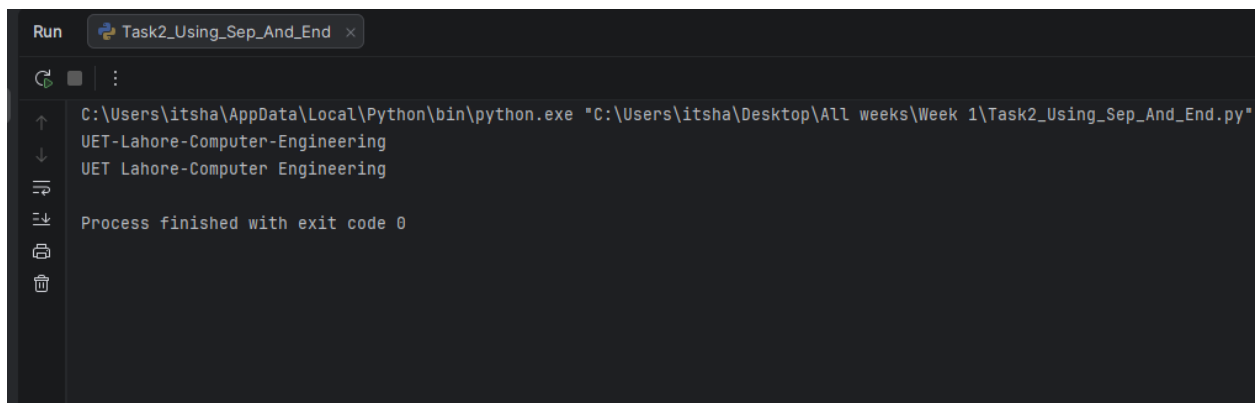
Code Summary:

This task shows how to use sep to specify a custom separator between multiple arguments and end to control what appears at the end of the print statement.

Key Code Lines:

```
# Task 2: Using sep and end
print("UET", "Lahore", "Computer", "Engineering", sep="-")
print("UET", "Lahore", end="-")
print("Computer", "Engineering")
```

Output:



```
Run Task2_Using_Sep_And_End x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week 1\Task2_Using_Sep_And_End.py"
UET-Lahore-Computer-Engineering
UET Lahore-Computer Engineering
Process finished with exit code 0
```

Task 3: Format Output with Escape Sequences

Write a program that uses escape sequences (\n) to format output across multiple lines in a single print statement.

Code Summary:

This task demonstrates using the newline escape sequence (\n) to create multi-line output from a single print() call.

Key Code Lines:

```
# Task 3: Format Output
print("Name: Hashir\nAge: 19\nCity: Lahore")
```

Output:

```
Run Task3_Format_Output x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week 1\Task3_Format_Output.py"
Name: Hashir
Age:19
City: Lahore
Process finished with exit code 0
```

Task 4: Take Name and Age Input

Write a program that takes user's name and age as input and displays them using type casting.

Code Summary:

This task demonstrates using the `input()` function with type casting. The name is kept as a string while the age is converted to an integer using `int()`.

Key Code Lines:

```
# Task 4: Take Name and Age
name = str(input("Enter your Name:"))
age = int(input("Enter your age:"))
print("Name: ", name)
print("Your age: ", age)
```

Output:

```
Run Task4_Type_Name_And_Age x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week 1\Task4_Type_Name_And_Age.py"
Enter your Name:Hashir
Enter your age:19
Name: Hashir
your age: 19
Process finished with exit code 0
```

Task 5: Identify Data Types

Write a program that declares variables of different data types and displays both their values and types using the type() function.

Code Summary:

This task demonstrates Python's built-in data types. Variables of type int, float, bool, and str are created, and the type() function is used to verify each variable's data type.

Key Code Lines:

```
# Task 5: Identify Data Types
x = 10
y = 3.14
z = True
name = "UET"
print(x)
print(y)
print(z)
print(type(x))
print(type(y))
print(type(z))
print(type(name))
```

Output:



```
Run Task5_Identify_Data_Types x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week 1\Task5_Identify_Data_Types.py"
10
3.14
True
<class 'int'>
<class 'float'>
<class 'bool'>
<class 'str'>
Process finished with exit code 0
```

Conclusions:

- Successfully understood and applied the print() function for output operations
- Learned to use sep and end parameters for custom formatting of output
- Mastered the use of escape sequences (\n) for multi-line formatting